DAY 5 REVISION NOTES

Functions, Variable Scope, and Assembling Project 1 (CLI Expense Tracker)

---

1. First Principles: Functions & The Call Stack

A **function** isolates logic into a reusable module with a strict interface: *Inputs (Parameters)* → *Processing* → *Outputs (Return Values)*. When a function is called, Python pauses global execution and pushes a new **Stack Frame (Local Scope)** onto the Call Stack in RAM.

Local Scope vs. Global Scope: Variables defined inside a function exist strictly within its stack frame. When the function finishes (`return`), its stack frame is popped from memory and local variables are erased. Global variables sit in top-level memory, but modifying global state inside functions creates hidden dependencies.

2. Memory Mechanics: Pass-by-Object-Reference

In Python, arguments are passed by **object reference**:
• **Immutable Objects (int, float, str, tuple):** Reassigning inside a function creates a new local reference without altering the caller's variable.
• **Mutable Objects (list, dict):** Passing a list into a function passes a reference to the exact same list in RAM. Calling `.append()` inside a function mutates the original object directly without needing to return it.

3. Entry Point Guard (__name__ == '__main__')

When a script is run directly (`python main.py`), Python sets `__name__ = '__main__'`. When imported by another module (`import main`), Python sets `__name__ = 'main'`. Wrapping your top-level loop in `if __name__ == '__main__':` ensures menu loops only run when invoked as a standalone script.

4. Project 1 Architecture (CLI Expense Tracker)

```
import json

def load_expenses():
    try:
        with open("expenses.json", "r") as f:
            return json.load(f)
    except FileNotFoundError:
        return []

def save_expenses(expenses):
    with open("expenses.json", "w") as f:
        json.dump(expenses, f)

def add_expense(expenses):
    item = input("Item: ").strip()
    while True:
        try:
            cost = float(input("Cost ($): ").strip().lstrip("$"))
            if cost > 0: break
        except ValueError:
            print("Enter a valid positive number.")
    expenses.append({"item": item, "cost": cost})
    save_expenses(expenses)

def main():
    expenses = load_expenses()
    # Interactive menu loop here...

if __name__ == "__main__":
    main()
```